Les Promesses en Node.js

De l'enfer des callbacks à async/await

Support de cours — Programmation asynchrone en JavaScript / Node.js

1. Introduction : pourquoi les promesses ?

Node.js repose sur un modèle d'entrées/sorties non bloquant : les opérations longues (lecture de fichiers, requêtes réseau, requêtes SQL) sont déléguées au système et leur résultat est notifié plus tard. Historiquement, ce modèle s'exprimait avec des callbacks : des fonctions passées en argument et invoquées au terme de l'opération.

Si les callbacks fonctionnent, ils souffrent de limitations structurelles qui rendent les applications réelles difficiles à écrire, lire et maintenir. Les promesses, introduites dans le standard ECMAScript 2015 (ES6) puis enrichies en ES2020 et ES2021, répondent directement à ces problèmes, et constituent aujourd'hui la brique de base sur laquelle reposent async/await.

1.1 Le problème n°1 : le « callback hell »

Dès qu'une opération asynchrone dépend du résultat de la précédente, les callbacks s'emboîtent les uns dans les autres. Le code s'enfonce vers la droite, formant la célèbre « pyramide de la mort » (pyramid of doom).

```
// Authentifier, puis charger le panier, puis le détail des produits, puis
payer...
login(credentials, (err, user) => {
  if (err) return console.error(err);
  getCart(user, (err, cart) => {
    if (err) return console.error(err);
    getProducts(cart, (err, products) => {
      if (err) return console.error(err);
      checkout(products, (err, receipt) => {
        if (err) return console.error(err);
        sendEmail(user, receipt, (err) => {
          if (err) return console.error(err);
          console.log("Commande terminée !");
        });
      });
    });
  });
});
```

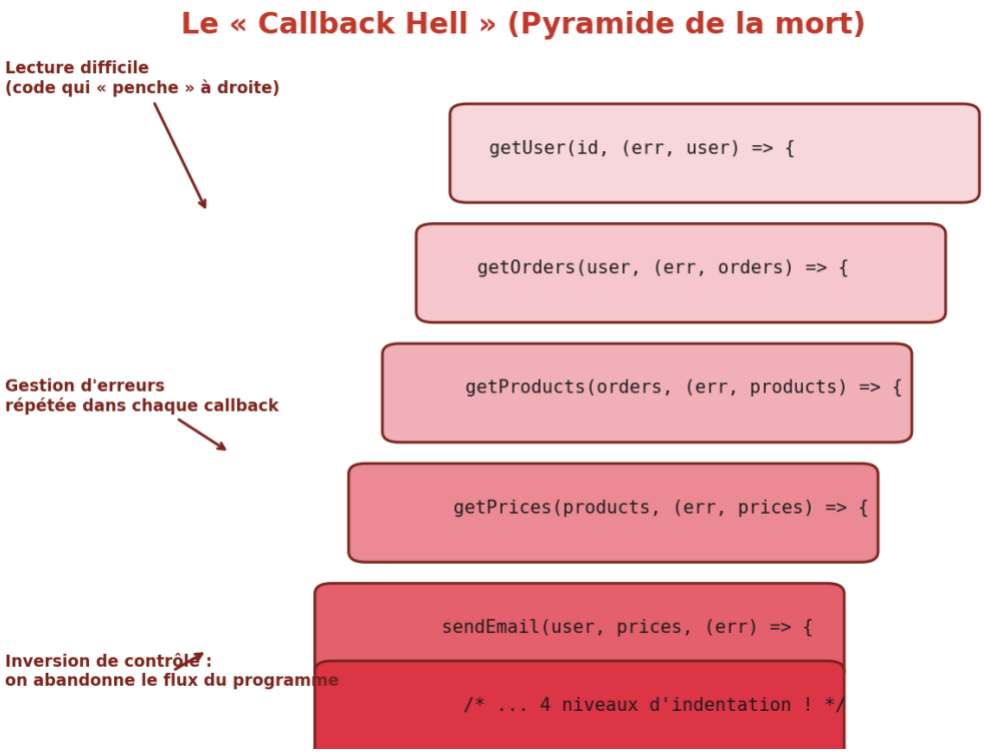


Figure 1 — Le code asynchrone par callbacks forme une pyramide difficile à lire.

- Chaque niveau ajoute un niveau d'indentation : la logique métier est éclatée et cachée.
- Impossible d'utiliser return ou throw de façon naturelle : tout transite par des arguments (err, result).
- Ajouter une étape au milieu impose de réindenter tout le bloc.

1.2 Le problème n°2 : la gestion d'erreurs fragile

Avec des callbacks, chaque fonction doit vérifier manuellement son paramètre err. Oublier une vérification, et l'erreur se propage silencieusement ou fait planter le processus. Il n'existe pas de mécanisme global : l'erreur « remonte » le long des imbrications, niveau par niveau, avec du code répétitif à chaque étage.

1.3 Le problème n°3 : l'inversion de contrôle

En passant un callback, on abandonne le contrôle du flux du programme à une autre fonction. On lui fait confiance pour : appeler le callback une seule fois, l'appeler avec les bons arguments, ne pas l'appeler trop tôt ou trop tard… Ce découplage fragilise le code et rend les bugs d'exécution (double appel, appel oublié) difficiles à déboguer.

> 💡 *Les promesses répondent à ces trois problèmes : un chaînage vertical (plus d'indentation), une propagation automatique des erreurs (catch centralisé), et une sémantique stricte garantissant qu'une promesse se résout au plus une fois.*

2. Définition et structure d'une promesse

Une promesse (Promise) est un objet qui représente la valeur disponible maintenant, dans le futur, ou jamais. Elle est le résultat « promis » d'une opération asynchrone : à l'instant où on la reçoit, on ne connaît pas encore le résultat, mais on dispose d'un objet standardisé permettant de s'y abonner.

Formellement, une promesse est créée par le constructeur suivant :

```
const promesse = new Promise((resolve, reject) => {
  // Corps de l'opération asynchrone
  // appeler resolve(valeur)  → succès
  // appeler reject(erreur)   → échec
});
```

2.1 Les composants d'une promesse

Une promesse combine quatre composants :

- L'exécuteur (executor) : la fonction (resolve, reject) => {...} passée au constructeur. Elle s'exécute immédiatement, de façon synchrone, à la création de la promesse.
- La fonction resolve(valeur) : à appeler pour marquer la promesse comme tenue. valeur peut être n'importe quelle valeur, y compris une autre promesse (résolution en chaîne).
- La fonction reject(raison) : à appeler pour marquer la promesse comme rompue. Par convention, raison est une instance d'Error.
- Les méthodes d'abonnement : then(onFulfilled, onRejected), catch(onRejected) et finally(onFinally) permettent de consommer le résultat. Elles retournent TOUTES une nouvelle promesse, d'où le chaînage.

2.2 Exemple complet : lire un fichier en promesse

```
const fs = require("fs");

// 1. Création : on emballe l'opération asynchrone dans une promesse
function lireFichier(chemin) {
  return new Promise((resolve, reject) => {
    fs.readFile(chemin, "utf8", (err, contenu) => {
      if (err) reject(err);          // échec → la promesse est rejetée
      else   resolve(contenu);       // succès → la promesse est tenue
    });
  });
}

// 2. Consommation : on s'abonne au résultat
lireFichier("config.json")
  .then((contenu) => console.log("Contenu :", contenu))
  .catch((err)   => console.error("Erreur :", err.message))
  .finally(()    => console.log("Lecture terminée (succès ou échec)"));
```

En pratique, on n'écrit presque jamais soi-même ce « wrapper » : la plupart des API de Node.js possèdent une version promesse, accessible via require('node:fs/promises') ou via util.promisify().

2.3 Le chaînage : la clé de voûte

Chaque appel à .then() retourne une nouvelle promesse. Si le callback retourne une valeur, la nouvelle promesse est tenue avec cette valeur ; s'il retourne une promesse, la nouvelle attend celle-ci ; s'il lève une erreur, la nouvelle est rejetée. C'est ce mécanisme qui permet d'écrire les opérations séquentielles à plat, sans indentation croissante :

```
getUser(42)                        // → Promise<User>
  .then(user    => getOrders(user))    // retourne une promesse → enchaînement
  .then(orders  => orders.map(o => o.id))
  .then(ids     => console.log("Commandes :", ids))
  .catch(err    => console.error("Une étape a échoué :", err.message));
// Un SEUL catch suffit : toute erreur intermédiaire s'y propage
automatiquement.
```

C'est la réponse directe au callback hell : le code se lit de haut en bas, chaque .then() correspond à une étape, et une seule gestion d'erreur centrale suffit.

3. Les états d'une promesse

Une promesse traverse exactement trois états au cours de sa vie. Comprendre ce cycle de vie est indispensable pour prévoir le comportement d'un programme et éviter les pièges classiques.

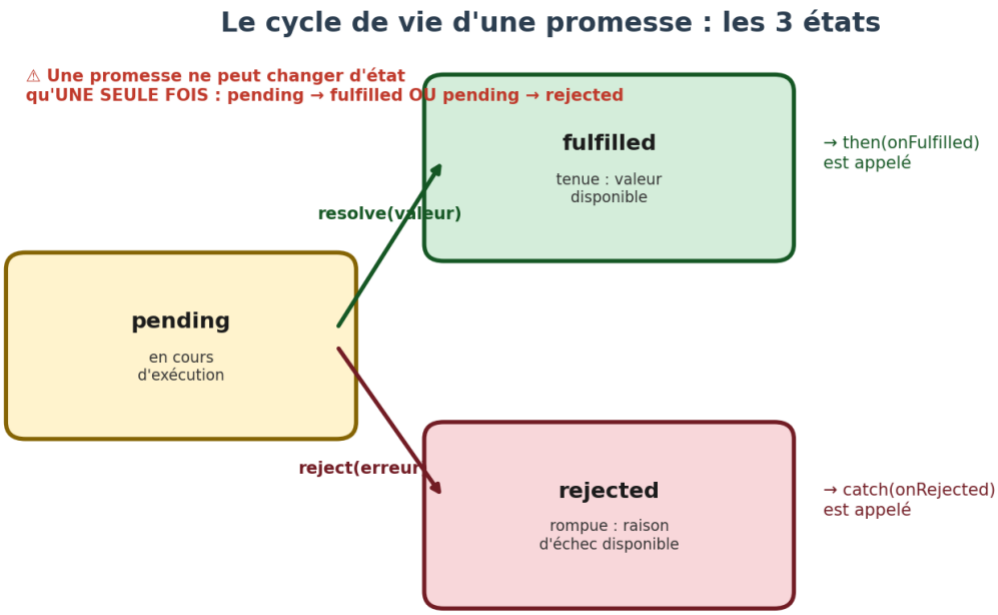


Figure 2 — Le cycle de vie : pending, fulfilled, rejected.

3.1 pending (en attente)

État initial. L'opération sous-jacente est en cours. La promesse n'a encore ni valeur ni raison d'échec. Tout appel à .then() / .catch() y accroche un rappel (callback) qui sera déclenché lors du changement d'état — jamais immédiatement si la promesse est déjà résolue (les rappels sont toujours asynchrones, cf. §5.2).

3.2 fulfilled (tenue)

L'opération a réussi. La promesse « contient » désormais sa valeur. Les callbacks passés à .then(onFulfilled) sont appelés, dans l'ordre d'enregistrement, avec cette valeur.

3.3 rejected (rompue)

L'opération a échoué. La promesse contient la raison de l'échec. Les callbacks passés à .catch() ou au second argument de .then() sont appelés avec cette raison.

- ⚠ Immutabilité : une fois fulfilled ou rejected, la promesse ne change plus JAMAIS d'état. Les appels ultérieurs à resolve()/reject() sont ignorés.
- ⚠ Exclusivité : fulfilled et rejected sont mutuellement exclusifs — une promesse ne peut pas réussir puis échouer, ni l'inverse.
- ⚠ Résolution unique : l'exécuteur est garanti d'être invoqué au plus une fois par chaque fonction (contrairement aux callbacks classiques).

```
const p = new Promise((resolve, reject) => {
  resolve("premier appel gagne");
  reject(new Error("ignoré"));     // sans effet : déjà tenue
  resolve("ignoré aussi");         // sans effet
});
p.then(v => console.log(v));       // → "premier appel gagne"
```

Point subtil : si une promesse A est résolue avec une promesse B, A « attend » B et adopte son état et sa valeur. On parle de résolution récursive (assimilation).

4. Les combinateurs de promesses

Un combinateur est une méthode statique de Promise qui agrège plusieurs promesses en une seule. Le choix du bon combinateur détermine le comportement face à l'échec et à la concurrence. Quatre combinateurs sont au programme : all, race, allSettled et any.

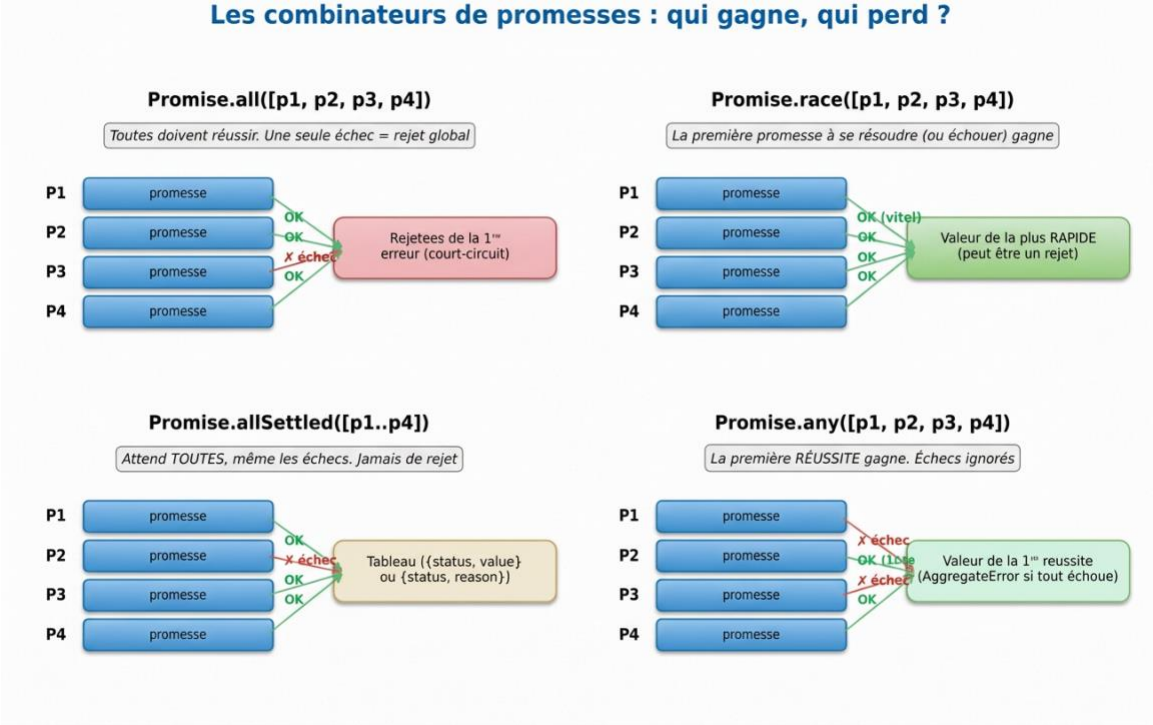


Figure 3 — Tableau comparatif : règle de résolution de chaque combinateur.

Combinateur	Règle de résolution	Rejette si…	Cas d'usage typique
Promise.all	Toutes réussissent → tableau des valeurs, dans l'ordre d'appel	Au moins une échoue (immédiatement)	Charger en parallèle des données dont on a TOUTES besoin
Promise.race	La première promesse (tenue ou rompue) impose son résultat	La plus rapide est un rejet	Timeout : Promise.race([requête, delai(5000)])
Promise.allSettled	Attend toutes → tableau d'objets {status, value|reason}	Jamais (ES2020)	Rapport de tâches indépendantes : certaines réussissent, d'autres non
Promise.any	La première réussite gagne → sa valeur	Toutes échouent → AggregateError (ES2021)	Serveur miroir : le premier qui répond correctement

4.1 Exemple détaillé : exécuter en parallèle

```
const { readFile } = require("node:fs/promises");

// Trois lectures LANCÉES EN PARALLÈLE (appels non bloquants)
const pA = readFile("a.txt", "utf8");
```

```
const pB = readFile("b.txt", "utf8");
const pC = readFile("c.txt", "utf8");

// all : attend que les trois soient terminées
Promise.all([pA, pB, pC])
  .then(([a, b, c]) => {           // déstructuration du tableau de résultats
    console.log("Total caractères :", a.length + b.length + c.length);
  })
  .catch(err => console.error("Une lecture a échoué :", err.message));

// ⚠ Piège fréquent — ne PAS écrire :
//   await pA; await pB; await pC;   → exécution SÉQUENTIELLE !
// Pour du parallélisme : lancer les promesses D'ABORD, combiner ENSUITE.
```

4.2 Exemple : Promise.race avec timeout

```
function delai(ms) {
  return new Promise((_, reject) =>
    setTimeout(() => reject(new Error("Délai dépassé")), ms));
}

// La requête et le sablier courent en parallèle : le premier « fini » gagne
Promise.race([
  fetch("https://api.example.com/data"),
  delai(5000)
])
  .then(res => res.json())
  .catch(err => console.error("Requête abandonnée :", err.message));
```

4.3 Exemple : any + allSettled

```
// any : premier miroir disponible
Promise.any([
  fetch("https://miroir1.example.com"),
  fetch("https://miroir2.example.com"),
  fetch("https://miroir3.example.com"),
]).then(res => console.log("Fourni par :", res.url));

// allSettled : rapport complet, jamais de rejet
Promise.allSettled([pA, pB, pC]).then(resultats => {
  for (const r of resultats) {
    if (r.status === "fulfilled") console.log("OK :", r.value);
    else                          console.log("Échec :", r.reason.message);
  }
});
```

5. Limites et inconvénients des promesses

Les promesses ne sont pas une solution miracle. Bien les comprendre suppose de connaître leurs zones d'ombre et leurs pièges classiques.

5.1 Impossibilité d'annulation

Une promesse lancée ne peut pas être annulée : il n'existe aucune méthode p.cancel(). Pour annuler une requête réseau ou une lecture de fichier, il faut passer par des mécanismes externes (AbortController pour fetch, timers gérés manuellement).

5.2 Les rappels restent asynchrones et peuvent surprendre

```
const p = Promise.resolve("valeur");
console.log("1. synchrone");
p.then(v => console.log("3. asynchrone :", v));
console.log("2. synchrone");       // → ordre réel : 1, 2, 3 (micro-tâche)
// Conséquence : un then() n'est JAMAIS exécuté immédiatement, même si la
promesse est déjà tenue.
```

5.3 Erreurs « avalées » et perte de contexte de pile

- Un .catch() mal placé peut masquer une erreur : si le catch lui-même ne relève pas, la chaîne continue en succès. Il faut réfléchir à chaque maillon.
- Le débogage est pénalisé : la pile d'appel est coupée à chaque frontière asynchrone (les erreurs remontent via les promesses, pas via la pile d'exécution). Les messages « UnhandledPromiseRejection » pointent parfois loin du vrai coupable.
- Oublier un .catch() sur la dernière promesse d'une chaîne déclenche un avertissement puis, à partir de Node 15, un arrêt brutal du processus.

5.4 Surcharge mémoire et limites de composition

- Chaque .then() crée un nouvel objet promesse : des chaînes très longues génèrent beaucoup d'objets intermédiaires (généralement négligeable, mais mesurable sur du gros volume).
- Combiner des centaines de milliers de promesses simultanées (Promise.all sur un énorme tableau) peut saturer la mémoire ; il faut alors une concurrence bornée (p-limit, p-map).
- L'exécuteur s'exécute immédiatement : une promesse est « chaude » (eager). Impossible de définir un calcul à la demande sans wrapper supplémentaire (contrairement aux observables).

Le point clé à retenir pour la suite : promesses et callbacks restent des styles à base de rappels. La syntaxe async/await ne change pas le modèle d'exécution — elle le rend lisible.

6. async et await : la syntaxe moderne

Introduits en ES2017, async et await sont un sucre syntaxique (sugar syntax) posé au-dessus des promesses : ils ne créent aucun nouveau mécanisme d'exécution. Une fonction async retourne toujours une promesse ; await suspend l'exécution de cette fonction jusqu'à la résolution d'une promesse, sans bloquer le thread.

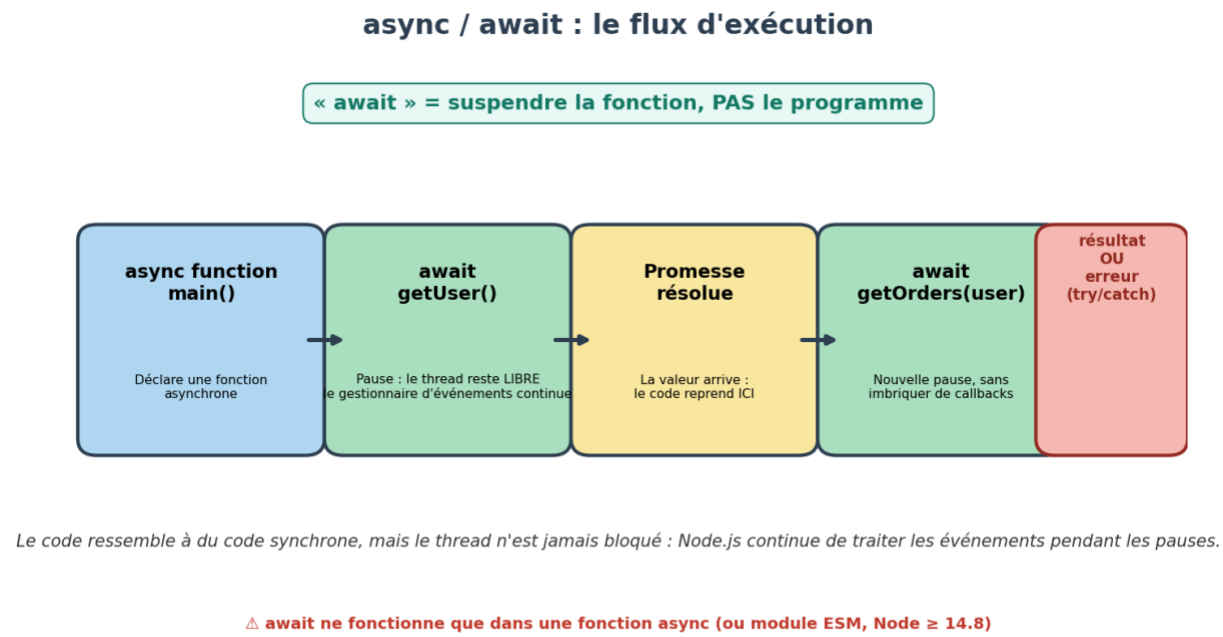


Figure 4 — await suspend la fonction, pas le programme : le thread reste libre.

6.1 Réécriture complète de l'exemple du panier

```
async function commander() {
  try {
    const user     = await login(credentials);        // pause, puis reprise
ici
    const cart     = await getCart(user);
    const products = await getProducts(cart);
    const receipt  = await checkout(products);
    await sendEmail(user, receipt);
    return receipt;                                    // la fonction retourne
une Promise<Receipt>
  } catch (err) {
    console.error("Échec de la commande :", err.message);
    throw err;                                         // re-propagation →
promesse rejetée
  } finally {
    console.log("Traitement terminé.");
  }
}
```

```
commander().then(r => console.log("OK", r.numero));
```

- await ne peut être utilisé que dans une fonction marquée async (ou, depuis Node 14.8 en ESM, au niveau racine du module : top-level await).
- await pause la fonction englobante et rend la main à la boucle d'événements : les autres requêtes continuent d'être servies.
- try/catch fonctionne enfin de façon naturelle autour du code asynchrone — c'était l'un des grands griefs contre les callbacks.

6.2 Parallélisme : ne pas await trop tôt

```
async function chargerConfig() {
  // ❌ SÉQUENTIEL : ~3 temps de réseau
  const db    = await lireConfigDB();
  const cache = await lireConfigCache();
  const env   = await lireEnv();

  // ✅ PARALLÈLE : ~1 temps de réseau (combinateurs toujours utiles !)
  const [db, cache, env] = await Promise.all([
    lireConfigDB(), lireConfigCache(), lireEnv(),
  ]);
}
```

6.3 Boucles asynchrones et cas piégeux

```
// Séquentiel (un fichier après l'autre)
for (const f of fichiers) {
  const contenu = await readFile(f, "utf8");
  traiter(contenu);
}
// ⚠ forEach NE comprend PAS await : les callbacks partent en parallèle non gérée
fichiers.forEach(async f => { await traiter(f); });   // à éviter
// Préférer : for...of (séquentiel) ou Promise.all(map(...)) (parallèle)
```

6.4 Comparaison finale

Critère	Callbacks	Promesses (.then)	async/await
Lisibilité	Faible (pyramide)	Bonne (chaînage vertical)	Excellente (style synchrone)
Gestion d'erreurs	Manuelle à chaque niveau	catch() centralisé	try/catch natif
Contrôle du flux	Inversion de contrôle	Résolution garantie unique	Identique aux promesses
Parallélisme	Complexe	Combinateurs (all/race…)	Combinateurs + await